```
• setUp: Run once for every test method to setup clean
data.
• Method: test _ false _ is _ true.
• setUp: Run once for every test method to setup clean
data.
• Method: test _ one _ plus _ one _ equals _ two.
• .
• ======================================================
===============
• FAIL: test _ false _ is _ true (catalog.tests.tests _ models.
YourTestClass)
• ------------------------------------------------------
-----------
• Traceback (most recent call last):
•   File "D:\Github\django _ tmp\library _ w _ t _ 2\locali-
brary\catalog\tests\tests _ models.py", line 22, in test _
false _ is _ true
•     self.assertTrue(False)
• AssertionError: False is not true
•
• ------------------------------------------------------
-----------
• Ran 3 tests in 0.075s
•
• FAILED (failures=1)
• Destroying the test database for alias 'default'.
```

Due to the assert function, we can exactly gauge the failure or success of any test function. As you can see here, that failure of a test is indicated through False. Here, the crucial factor remains to be the descriptive names of objects you should put in the code because they will help you identify the failed tests.

Now, let's see how you can carry out specific tests.

Specific Tests

Models

Let's take an example of the Author model here. We can test all the fields with a design that represents these values. If we don't test the values, there is no way to know how the fields perform.